

claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_793a5d86-1c4\agent-a6d9d7f8057646008.jsonl`
- SHA-256: `93d77cc9e4ee8ae37e6c6acdcf00bdfc674b96779561be46a2e75c7f7ff4350e`
- Source modified: `2026-07-02T08:17:51+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_793a5d86-1c4`
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`
```

Transcript

1. user (2026-07-02T08:16:24.737Z)

You are reviewing OPEN GitHub PR #418 on Khelsutra/badminton-highlight-indexer to decide its fate: should we TAKE IT OVER (adopt/rebase/verify/merge) or CLOSE it as superseded by already-merged work?

PR #418: "fix(api): /api/source downloads remote source ONCE into reusable cache (R1-03)" (head branch: origin/fix/api-source-remote-cache, mergeable=CONFLICTING)
Investigation hint: Adds _acquire_source_for_serving: fetch remote source once into <output>/source_cache/<video_id>/<name>, atomic publish, reuse across Range requests. STRONG hypothesis: SUPERSEDED by merged #425 (P3 source caching) which did the same thing. CHECK: does master backend/main.py already have source_cache / a fetch-once-and-reuse path for /api/source? Compare #418 diff intent vs what #425 actually landed. Decide: fully superseded (close), or does #418 add something #425 lacks?

Already MERGED onto master (verify against actual master code, do not assume):

```
- #422 = fix for --output-dir clobbering config.json output.output_dir (CODE_MAP updated for it)
- #423 = docs: health-remediation plan added + DOC_STATUS/README/NEXT_STEPS registered; audit marked snapshot
- #424 = P2: /api/process idempotent by content hash (#340 re-bill fix)
- #425 = P3: /api/source resolves remote source into output-scoped source_cache, reuses across Range requests, warms annotation proxy
- #426 = P4: bounded ffmpeg/ffmpeg subprocess timeouts on serving paths
- #427 = P5: clean remote-input scratch dirs after process/compile/assets/CLI; durable reports
- #428 = P1/P6: backend/storage/gcs.py lazy-import mypy typing fix
master HEAD = d00b4db "fix gcs storage lazy import typing (#428)"
```

Working dir: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (currently on master @ d00b4db). This is a SHARED checkout used by sibling sessions ? DO NOT checkout branches, DO NOT modify the working tree, DO NOT run git merge/rebase/add/commit. Read-only investigation only.

Do this:

1. Get the PR's intended change: run `gh pr diff 418 -R Khelsutra/badminton-highlight-indexer` (read the full diff). Also `gh pr view 418 -R Khelsutra/badminton-highlight-indexer --json body,files,statusCheckRollup` if useful.
2. Read the CURRENT master version of every file the PR touches (Read / Grep the files under C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer). To see the PR's branch version of a file without checkout, use: `git -C "C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer" show origin/fix/api-source-remote-cache:<path>`.
3. Determine whether master ALREADY implements the PR's intent (superseded) or still lacks it. Cite concrete master file:line evidence either way. Do not trust the hint or handoff ? verify in code.
4. If CONFLICTING, explain precisely why (which files/hunks) and whether the conflict is because master already did the same thing (=> superseded/close) or because of adjacent edits (=>

rebasable).

5. Judge correctness/quality of the PR change itself.

6. Recommend a fate:

- CLOSE_SUPERSEDED: master already implements the intent; the PR adds nothing material.
- TAKE_OVER_CLEAN: mergeable & still-needed; adopt and verify as-is.
- TAKE_OVER_REBASE: still-needed but conflicting; needs a rebase onto master + re-verify, then merge.
- OWNER_DECISION: product/strategy/ambiguous ? needs the owner.

Return ONLY the structured object. Be precise and cite master file:line in master_evidence.

2. assistant (2026-07-02T08:16:28.096Z)

I'll investigate PR #418 to determine its fate. Let me start by getting the PR's diff and details, and checking the current master state.

3. user (2026-07-02T08:16:31.661Z)

```
diff --git a/backend/main.py b/backend/main.py
index 99246fe..078fa2b 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -620,13 +620,79 @@ def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
     _proxy_inflight.discard(video_id)

+# Per-video locks so concurrent FIRST requests for a cold remote source serialize on ONE
download
+# (the losers re-check and reuse the winner's copy) instead of each pulling the full object.
+_source_fetch_locks: "dict[str, threading.Lock]" = {}
+_source_fetch_locks_guard = threading.Lock()
+
+
+def _source_cache_dir(cfg) -> str:
+    """Where /api/source caches fetched REMOTE sources ? under the guarded local output
root."""
+    return os.path.join(output_base(cfg), "source_cache")
+
+
+def _acquire_source_for_serving(video_id: str, raw_ref: str, cfg) -> str:
+    """Resolve a video's recorded filepath to the LOCAL file ``/api/source`` streams.
+
+    A local path is an identity passthrough (no copy, no temp ? byte-identical to serving the
+    recorded path directly). A REMOTE ref (``gs://``/``s3://``/``gdrive://``?) is fetched ONCE
+    into
+    ``<output>/source_cache/<video_id>/<name>`` and REUSED: ``FileResponse`` serves Range, so
+    one
+    scrubbing ``<video>`` tag issues many independent requests to this URL ? before this cache
+    each
+    request re-downloaded the full object into a fresh, never-deleted temp dir (a cloud-egress
+    bill
+    plus a disk-fill of the appliance; audit R1-03).
+
+    Concurrency + crash safety: the download goes to a unique ``*.partial`` name inside the
+    cache
+    dir and is published with an atomic ``os.replace`` ? a truncated file can never sit at the
+    served name, and a failed fetch leaves nothing behind. Racing first requests serialize on a
+    per-video lock. The download is free-space guarded (D11) when the backend can ``stat`` the
+    object's size; the cached copy is a regenerable intermediate (safe to delete any time)."""
+    settings = _storage_settings(cfg)
+    ref = storage.resolve_input_ref(raw_ref, settings)
+    if ref.backend == "local":
+        return ref.key # identity ? today's local behaviour, byte-for-byte
+
+    vid_dir = os.path.join(_source_cache_dir(cfg), safe_output_filename(video_id))
```

```

+     try:
+         fname = safe_output_filename(ref.name or "in.mp4")
+     except ValueError: # a hostile/odd remote object name never dictates the local filename
+         fname = "in.mp4"
+     final = os.path.join(vid_dir, fname)
+     if os.path.isfile(final):
+         return final # cache hit ? no re-download, whatever the browser's Range pattern
+
+     with _source_fetch_locks_guard:
+         lock = _source_fetch_locks.setdefault(video_id, threading.Lock())
+     with lock:
+         if os.path.isfile(final):
+             return final # a concurrent first request fetched it while we waited on the lock
+         os.makedirs(vid_dir, exist_ok=True)
+         need = None
+         try: # sizing is best-effort ? a backend that can't stat still fetches (and fails
loudly)
+             need = int(storage.get_storage(ref.backend, config=settings).stat(ref).size)
+         except Exception: # noqa: BLE001 - stat is optional; require_free_bytes(None) is a no-
op
+             need = None
+         require_free_bytes(vid_dir, need, stage="source-cache download")
+         tmp = os.path.join(vid_dir, f".{fname}.{uuid.uuid4().hex}.partial")
+         try:
+             storage.fetch(ref.uri, tmp, config=settings)
+             os.replace(tmp, final) # atomic publish ? readers only ever see a COMPLETE file
+         finally:
+             if os.path.exists(tmp): # fetch (or publish) failed ? never leave a partial behind
+                 try:
+                     os.remove(tmp)
+                 except OSError:
+                     pass
+         return final
+
+
+ @app.get("/api/source/{video_id}")
+ def get_source(video_id: str, request: Request):
+     """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub. Serves
a
+     frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels still
keyed
+     to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy in the
-     background for next time ? non-blocking, so the first load is never delayed. Proxy warming
is
+     LOCAL-source only (remote-source proxying is a follow-up)."""
+     background for next time ? non-blocking, so the first load is never delayed. A REMOTE
source is
+     downloaded ONCE into ``<output>/source_cache/<video_id>/`` and every later request (each
browser
+     Range request hits this URL anew) streams that local copy ? never a re-download (R1-03)."""
+     rec = request.app.state.db.get_video(video_id)
+     if not rec or not rec.get("filepath"):
+         raise HTTPException(status_code=404, detail="Video not found.")
@@ -641,20 +707,21 @@ def get_source(video_id: str, request: Request):
+     if proxy_path and os.path.isfile(proxy_path):
+         return FileResponse(proxy_path, media_type="video/mp4")
+
+     # 2) Otherwise resolve + serve the source.
+     # 2) Otherwise resolve + serve the source (remote ? the reused source-cache copy).
+     try:
+         local = storage.acquire_local_input(rec["filepath"], getattr(cfg, "storage", None))
+         is_local = storage.input_is_local(rec["filepath"], getattr(cfg, "storage", None))
+     except ValueError as e: # unsupported scheme etc. ? a clean client error, never a 500
+         local = _acquire_source_for_serving(video_id, rec["filepath"], cfg)
+     except HTTPException:

```

```

+         raise # already a clean client answer (e.g. the 507 free-space guard)
+     except ValueError as e: # unsupported scheme / unsafe id ? a clean client error, never a
500
+         raise HTTPException(status_code=400, detail=str(e)) from e
+     except Exception as e: # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced (not
swallowed)
+         raise HTTPException(status_code=502, detail=f"Could not resolve the source video: {e}")
from e
+     if not os.path.isfile(local):
+         raise HTTPException(status_code=404, detail="Source video file not found.")

-     # 3) Warm a proxy in the background for next time (local source only).
-     if is_local:
-         threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True).start()
+     # 3) Warm a proxy in the background for next time. Remote sources now warm too ? the
+     #     source-cache copy is a plain local file (closes the 'remote-source proxying' follow-
up).
+     threading.Thread(target=_warm_annotation_proxy, args=(video_id, local, cfg),
daemon=True).start()

+     media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
+     return FileResponse(local, media_type=media)
diff --git a/tests/test_api_annotations.py b/tests/test_api_annotations.py
index c212810..cfe96d8 100644
--- a/tests/test_api_annotations.py
+++ b/tests/test_api_annotations.py
@@ -2,6 +2,9 @@
GET /api/source/{video_id} (serve a video's source for annotation) and
POST /api/annotations (persist finished rally labels as the byte-verbatim annotator CSV)."""

+import threading
+import time
+
+import pytest

+pytest.importorskip("httpx") # TestClient needs httpx; skip cleanly where absent
@@ -10,6 +13,7 @@
import backend.main as m
from backend.config.models import AppConfig
from backend.infrastructure.database import Database
+from backend.storage.ref import StorageStat

class _InlineExecutor:
@@ -64,6 +68,151 @@ def test_source_serves_the_proxy_when_one_is_ready(tmp_path, monkeypatch):
    assert r.content == b"PROXY-bytes" # proxy wins over source

+## ---- GET /api/source/{video_id} ? REMOTE-backed sources (R1-03: download ONCE, reuse) ----
+_REMOTE = b"remote-bytes"
+
+
+
+class _StubBackend:
+    """A stat-capable storage stub so the free-space guard can size the object without any
SDK."""
+
+    def __init__(self, size=None):
+        self._size = len(_REMOTE) if size is None else size
+
+    def stat(self, ref):
+        return StorageStat(size=self._size)
+
+
+def _counting_fetch(counter, payload=_REMOTE, delay=0.0):

```

```

+ """A storage.fetch double that counts downloads and writes ``payload`` to ``dst``."""
+
+ def _fetch(uri, dst=None, *, config=None, overwrite=False):
+     counter["n"] += 1
+     counter["uri"] = uri
+     if delay:
+         time.sleep(delay)
+     with open(dst, "wb") as fh:
+         fh.write(payload)
+     return str(dst)
+
+ return _fetch
+
+
+def test_source_remote_is_fetched_once_and_reused(tmp_path, monkeypatch):
+    """R1-03: FileResponse serves Range, so a scrubbing browser hits this URL many times ? the
+    remote object must be downloaded ONCE into the source cache and served from there after."""
+    client, db = _client(tmp_path, monkeypatch)
+    db.add_video("beefcafe", "gs://bucket/videos/match.mp4", "processed", [])
+    counter = {"n": 0}
+    monkeypatch.setattr(m.storage, "fetch", _counting_fetch(counter))
+    monkeypatch.setattr(m.storage, "get_storage", lambda *a, **k: _StubBackend())
+
+    r1 = client.get("/api/source/beefcafe")
+    r2 = client.get("/api/source/beefcafe", headers={"Range": "bytes=0-3"})
+    assert r1.status_code == 200 and r1.content == _REMOTE
+    assert r2.status_code == 206 and r2.content == _REMOTE[:4] # Range served from the cache
+    assert counter["n"] == 1 # ONE download total, both requests answered
+    assert counter["uri"] == "gs://bucket/videos/match.mp4"
+    cached = tmp_path / "source_cache" / "beefcafe" / "match.mp4"
+    assert cached.read_bytes() == _REMOTE # cached under output_base, keyed by video_id
+
+
+def test_source_local_path_stays_identity_passthrough(tmp_path, monkeypatch):
+    """A local-path video is served straight off its recorded path ? no fetch, no cache dir."""
+    client, db = _client(tmp_path, monkeypatch)
+    vid = tmp_path / "match.mp4"
+    vid.write_bytes(b"local-bytes")
+    db.add_video("localvid", str(vid), "processed", [])
+
+    def _boom(*a, **k):
+        raise AssertionError("fetch must not be called for a local source")
+
+    monkeypatch.setattr(m.storage, "fetch", _boom)
+
+    r = client.get("/api/source/localvid")
+    assert r.status_code == 200 and r.content == b"local-bytes"
+    assert not (tmp_path / "source_cache").exists() # nothing was copied or cached
+
+
+def test_source_remote_stale_partial_is_never_served(tmp_path, monkeypatch):
+    """A crashed/in-flight download leaves only ``*.partial`` names; the endpoint must never
+    serve one ? it re-fetches and atomically publishes the complete copy."""
+    client, db = _client(tmp_path, monkeypatch)
+    db.add_video("cafe0123", "gs://bucket/videos/match.mp4", "processed", [])
+    vid_dir = tmp_path / "source_cache" / "cafe0123"
+    vid_dir.mkdir(parents=True)
+    (vid_dir / ".match.mp4.deadbeef.partial").write_bytes(b"TRUNCATED")
+    counter = {"n": 0}
+    monkeypatch.setattr(m.storage, "fetch", _counting_fetch(counter))
+    monkeypatch.setattr(m.storage, "get_storage", lambda *a, **k: _StubBackend())
+
+    r = client.get("/api/source/cafe0123")
+    assert r.status_code == 200
+    assert r.content == _REMOTE # the complete fetch ? never the truncated partial

```

```

+     assert counter["n"] == 1
+     assert (vid_dir / "match.mp4").read_bytes() == _REMOTE
+
+
+def test_source_remote_fetch_failure_is_502_and_leaves_no_partial(tmp_path, monkeypatch):
+    """A failed download surfaces as 502 and cleans its partial, so a retry starts clean."""
+    client, db = _client(tmp_path, monkeypatch)
+    db.add_video("fail0123", "gs://bucket/videos/match.mp4", "processed", [])
+    monkeypatch.setattr(m.storage, "get_storage", lambda *a, **k: _StubBackend())
+
+    def _broken(*a, **k):
+        raise RuntimeError("bucket unreachable")
+
+    monkeypatch.setattr(m.storage, "fetch", _broken)
+    assert client.get("/api/source/fail0123").status_code == 502
+    vid_dir = tmp_path / "source_cache" / "fail0123"
+    assert not any(vid_dir.iterdir()) # no partial (or anything else) left behind
+
+    counter = {"n": 0}
+    monkeypatch.setattr(m.storage, "fetch", _counting_fetch(counter))
+    r = client.get("/api/source/fail0123") # the retry succeeds and caches normally
+    assert r.status_code == 200 and r.content == _REMOTE and counter["n"] == 1
+
+
+def test_source_remote_download_is_free_space_guarded(tmp_path, monkeypatch):
+    """D11: an object the disk can't hold is refused with 507 BEFORE any bytes are pulled."""
+    client, db = _client(tmp_path, monkeypatch)
+    db.add_video("big00001", "gs://bucket/videos/huge.mp4", "processed", [])
+    monkeypatch.setattr( # ~1 EB ? larger than any test box's free space
+        m.storage, "get_storage", lambda *a, **k: _StubBackend(size=10**18)
+    )
+
+    def _boom(*a, **k):
+        raise AssertionError("must not fetch when the disk cannot hold the object")
+
+    monkeypatch.setattr(m.storage, "fetch", _boom)
+    assert client.get("/api/source/big00001").status_code == 507
+
+
+def test_source_concurrent_first_requests_download_once(tmp_path, monkeypatch):
+    """Two racing first requests serialize on the per-video lock: ONE download, both served."""
+    cfg = AppConfig()
+    cfg.output.output_dir = str(tmp_path)
+    counter = {"n": 0}
+    monkeypatch.setattr(m.storage, "fetch", _counting_fetch(counter, delay=0.2))
+    monkeypatch.setattr(m.storage, "get_storage", lambda *a, **k: _StubBackend())
+
+    results = []
+
+    def _hit():
+        results.append(
+            m._acquire_source_for_serving("race0001", "gs://bucket/videos/match.mp4", cfg)
+        )
+
+    threads = [threading.Thread(target=_hit) for _ in range(2)]
+    for t in threads:
+        t.start()
+    for t in threads:
+        t.join()
+
+    assert counter["n"] == 1 # the loser waited, re-checked, and reused the winner's copy
+    assert len(results) == 2 and results[0] == results[1]
+    with open(results[0], "rb") as fh:
+        assert fh.read() == _REMOTE
+
+
+

```

```
# ---- POST /api/annotations ----
def test_annotations_persist_verbatim_csv(tmp_path, monkeypatch):
    client, _ = _client(tmp_path, monkeypatch)
```

4. user (2026-07-02T08:16:33.162Z)

```
{"body": "## What\n\nFixes audit finding **R1-03**: `GET /api/source/{video_id}` called  

`storage.acquire_local_input` on **every** request, and for a REMOTE-backed video (`gs://` /  

`s3://` / `gdrive://`) that meant a fresh `tempfile.mkdtemp(prefix=\"rally-input-\")` + a  

**full object download per request** ? never deleted. `FileResponse` serves Range, so one  

browser scrubbing a `` tag issues many independent requests to the same URL: ~10 seeks on  

a 4 GB match ? ~40 GB of bucket egress **and** ~40 GB of orphaned temp dirs, with every seek  

stalling behind a multi-GB download.\n\nNow  

(`backend/main.py::_acquire_source_for_serving`):\n\n- A remote source is fetched **once** into  

`<output>/source_cache/<video_id>/<name>` and every later request streams that local copy.\n-  

**Atomic publish**: the download goes to a unique `.<name>.<uuid>.partial` in the cache dir,  

then `os.replace` onto the served name ? a truncated file can never sit at the served path, and  

a failed fetch cleans its partial (502 surfaced, retry starts clean). No `finally`-delete of the  

served file ? `FileResponse` streams after the handler returns.\n- **Concurrent first requests**  

serialize on a per-video lock; losers re-check under the lock and reuse the winner's copy (one  

download total).\n- The download is **free-space guarded** (`require_free_bytes`, D11 / #407)  

when the backend can `stat` the object size ? refused with 507 *before* any bytes move; the  

cache lives under the guarded output root and is a regenerable intermediate (safe to delete any  

time).\n- **Local-path videos stay a byte-identical identity passthrough** ? no copy, no cache  

dir, `storage.fetch` never called; path-safety unchanged (`safe_output_filename` on the video_id  

path component + a sanitized cached filename).\n- **Proxy warming now covers remote sources  

too** (the cached copy is a plain local file), closing the `\"remote-source proxying is a follow-  

up\"` note from #406 ? once the D4 proxy exists it wins as before.\n\n## Why\n\nDirect cloud-  

egress cost leak + a disk-fill denial-of-service of the appliance, reachable through a supported  

flow (remote-registered assets via P4 `/api/assets` + the P8 annotate-in-UI player). See  

`AUDIT_REPORT_khelsutra-guru_2026-07-02.md` R1-03 evidence + verifier notes.\n\n##  

Verification\n\nRan locally (Windows, Python 3.13):\n- `python -m pytest  

tests/test_api_annotations.py -q` ? **12 passed** (6 new tests: remote fetched once across two  

requests with Range served from cache; local identity passthrough with fetch forbidden; stale  

`.partial` never served; failed fetch ? 502 and no partial left; 507 free-space refusal before  

fetch; two-thread cold race downloads once).\n- `python -m pytest tests/test_input_source.py  

tests/test_api_cloud_dispatch.py tests/test_storage.py -q` ? **81 passed** (adjacent  

storage/input behavior unchanged).\n- `ruff check backend/main.py tests/test_api_annotations.py`  

? clean.\n- `python -m mypy backend/main.py tests/test_api_annotations.py` ? no issues (note  

`backend.main` is mypy-quarantined via `mypy.ini`).\n\nNot verified locally: a real bucket  

download (network is mocked per repo test policy ? offline suite only); full suite + coverage  

floor is CI's job.\n\n? Generated with [Claude Code](https://claude.com/claude-code)", "files": [{  

"path": "backend/main.py", "additions": 76, "deletions": 9, "changeType": "MODIFIED"}, {"path": "tests/te  

st_api_annotations.py", "additions": 149, "deletions": 0, "changeType": "MODIFIED"}], "mergeable": "CONF  

LICTING", "state": "OPEN", "statusCheckRollup": [{"__typename": "CheckRun", "completedAt": "2026-07-  

02T04:56:03Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-  

highlight-indexer/actions/runs/28566436129/job/84694593906", "name": "Ruff lint", "startedAt": "2026  

-07-  

02T04:55:55Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "  

2026-07-  

02T04:56:02Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-  

highlight-indexer/actions/runs/28566436129/job/84694593962", "name": "Leak scan (PII / attribution  

guard)", "startedAt": "2026-07-  

02T04:55:55Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "  

2026-07-  

02T04:56:00Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-  

highlight-indexer/actions/runs/28566436129/job/84694593926", "name": "Markdown link check (live do  

cs)", "startedAt": "2026-07-  

02T04:55:55Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "  

2026-07-  

02T04:56:29Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-  

highlight-indexer/actions/runs/28566436129/job/84694593946", "name": "Mypy (lenient baseline)", "st  

artedAt": "2026-07-  

02T04:55:55Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "  

2026-07-
```

```

02T04:56:19Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566436129/job/84694593924", "name": "Syntax sweep + import smoke (no GPU stack)", "startedAt": "2026-07-
02T04:55:55Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "2026-07-
02T04:58:47Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566436129/job/84694593929", "name": "Full test suite (offline, mocked)", "startedAt": "2026-07-
02T04:55:56Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "2026-07-
02T04:56:27Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566436129/job/84694593928", "name": "Golden fixtures cross-OS (ubuntu-latest)", "startedAt": "2026-07-
02T04:55:57Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "2026-07-
02T04:56:59Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566436129/job/84694593990", "name": "Golden fixtures cross-OS (windows-latest)", "startedAt": "2026-07-
02T04:55:56Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "2026-07-
02T04:56:10Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566436129/job/84694593941", "name": "Golden fixtures cross-OS (macos-latest)", "startedAt": "2026-07-
02T04:55:55Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "2026-07-
02T04:56:26Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566436129/job/84694593912", "name": "Dependency license scan (no copyleft in runtime deps)", "startedAt": "2026-07-
02T04:55:55Z", "status": "COMPLETED", "workflowName": "CI"}, {"__typename": "CheckRun", "completedAt": "2026-07-
02T04:56:58Z", "conclusion": "SUCCESS", "detailsUrl": "https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28566436129/job/84694593950", "name": "Built wheel smoke (install + launch)", "startedAt": "2026-07-02T04:55:55Z", "status": "COMPLETED", "workflowName": "CI"}]}

```

5. assistant (2026-07-02T08:16:36.558Z)

Now let me examine the current master version of the files this PR touches. Let me read the relevant sections of master's `backend/main.py` and check the merged #425.

6. user (2026-07-02T08:16:38.713Z)

```

d00b4db fix gcs storage lazy import typing (#428)
19ebe5a clean remote input scratch dirs (#427)
c6bb4bf bound ffmpeg subprocess timeouts
6c767b5 cache remote source material
9523d17 fix process idempotency cache
e517475 docs: track khelsutra-guru health remediation plan
4f148d1 fix: config-correctness + typing fixes (audit R1-19, R1-20, R1-27) (#422)
d80aa02 docs: truth-sync ? audit report + fix stale state in CLAUDE.md, BACKLOG, DOC_STATUS, README, NEXT_STEPS (#421)

```

7. user (2026-07-02T08:16:39.569Z)

```

600:_source_cache_locks: Dict[str, threading.Lock] = {}
601:_source_cache_locks_guard = threading.Lock()
608:def _source_cache_dir(cfg) -> str:
609:    return os.path.join(output_base(cfg), "source_cache")
638:def _source_cache_lock(video_id: str) -> threading.Lock:
639:    with _source_cache_locks_guard:
640:        return _source_cache_locks.setdefault(video_id, threading.Lock())
643:def _source_cache_path(video_id: str, input_ref, cfg) -> str:
647:    cache_dir = _source_cache_dir(cfg)
654:    cache_path = _source_cache_path(video_id, input_ref, cfg)
657:    lock = _source_cache_lock(video_id)

```

8. assistant (2026-07-02T08:16:42.239Z)

Master already has `source\_cache` machinery from #425 (commit `6c767b5` cache remote source material`). Let me read the full master implementation to compare against PR #418's intent.

9. user (2026-07-02T08:16:43.329Z)

```
590         return FileResponse(path, media_type=media, filename=name)
591
592
593     # --- Annotate-in-UI (P8): serve a video's source + persist finished rally labels
594     -----
595
596     # In-flight annotation-proxy generations ? dedup so a browser's many Range requests to
597     /api/source
598     # don't each kick off a transcode.
599     _proxy_inflight: "set[str]" = set()
600     _proxy_lock = threading.Lock()
601     _source_cache_locks: Dict[str, threading.Lock] = {}
602     _source_cache_locks_guard = threading.Lock()
603
604     def _proxies_dir(cfg) -> str:
605         return os.path.join(output_base(cfg), "proxies")
606
607
608     def _source_cache_dir(cfg) -> str:
609         return os.path.join(output_base(cfg), "source_cache")
610
611
612     def _report_config_for_input(config: Any, raw_input: str) -> Any:
613         """Return report config that keeps remote-input reports out of disposable
614         scratch."""
615         storage_config = (
616             config.get("storage") if isinstance(config, dict) else getattr(config,
617 "storage", None)
618         )
619         try:
620             if storage.input_is_local(raw_input, storage_config):
621                 return config
622         except ValueError:
623             return config
624         try:
625             cfg = (
626                 config.model_dump(mode="json")
627                 if hasattr(config, "model_dump")
628                 else dict(config or {})
629             )
630         except Exception:
631             cfg = {}
632         report = dict(cfg.get("report") or {})
633         if report.get("output_dir"):
634             return config
635         report["output_dir"] = os.path.join(output_base(config), "reports")
636         cfg["report"] = report
637         return cfg
638
639     def _source_cache_lock(video_id: str) -> threading.Lock:
640         with _source_cache_locks_guard:
641             return _source_cache_locks.setdefault(video_id, threading.Lock())
642
643
644     def _source_cache_path(video_id: str, input_ref, cfg) -> str:
```

```

644     ext = os.path.splitext(input_ref.name or "")[1].lower()
645     if not ext or "/" in ext or "\\" in ext or len(ext) > 16:
646         ext = ".mp4"
647     cache_dir = _source_cache_dir(cfg)
648     name = safe_output_filename(f"{video_id}{ext}")
649     return resolve_under_root(os.path.join(cache_dir, name), cache_dir)
650
651
652 def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
653     """Fetch a remote source once into an output-scoped source/proxy cache."""
654     cache_path = _source_cache_path(video_id, input_ref, cfg)
655     if os.path.isfile(cache_path):
656         return cache_path
657     lock = _source_cache_lock(video_id)
658     with lock:
659         if os.path.isfile(cache_path):
660             return cache_path
661         cache_dir = os.path.dirname(cache_path)
662         os.makedirs(cache_dir, exist_ok=True)
663         need = _remote_input_size_bytes(input_ref, getattr(cfg, "storage", None))
664         require_free_bytes(cache_dir, need, stage="source-cache")
665         tmp = os.path.join(
666             cache_dir, f".{os.path.basename(cache_path)}.{uuid.uuid4().hex}.tmp"
667         )
668         try:
669             storage.fetch(input_ref.uri, tmp, config=_storage_settings(cfg))
670             os.replace(tmp, cache_path)
671         finally:
672             try:
673                 if os.path.exists(tmp):
674                     os.remove(tmp)
675             except OSError:
676                 pass
677         return cache_path
678
679
680 def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
681     """Background, best-effort: generate the annotation proxy ONCE so subsequent
682     /api/source loads
683     serve the lighter, seekable copy. Deduped; never blocks the request; the source
684     keeps serving if
685     it fails."""
686     with _proxy_lock:
687         if video_id in _proxy_inflight:
688             return
689         _proxy_inflight.add(video_id)
690     try:
691         from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
692         if not needs_proxy(src_local):
693             return # already light enough ? no proxy worth making
694         make_annotation_proxy(src_local, os.path.join(_proxies_dir(cfg),
695             f"{video_id}.mp4"))
696     except Exception as e: # noqa: BLE001 - warming is best-effort
697         logger.warning("proxy: warm failed for %s: %s", video_id, e)
698     finally:
699         with _proxy_lock:
700             _proxy_inflight.discard(video_id)
701
702 @app.get("/api/source/{video_id}")
703 def get_source(video_id: str, request: Request):
704     """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub.
705     Serves a
706     frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels
707     still keyed

```

```

704         to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy
in the
705         background for next time ? non-blocking, so the first load is never delayed. Remote
sources are
706         fetched once into an output-scoped source cache, then served and proxied from that
stable copy. """
707         rec = request.app.state.db.get_video(video_id)
708         if not rec or not rec.get("filepath"):
709             raise HTTPException(status_code=404, detail="Video not found.")
710         cfg = request.app.state.config
711
712         # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
713         try:
714             proxy_name = safe_output_filename(f"{video_id}.mp4")
715             proxy_path = resolve_under_root(os.path.join(_proxies_dir(cfg), proxy_name),
716             _proxies_dir(cfg))
717         except ValueError:
718             proxy_path = ""
719         if proxy_path and os.path.isfile(proxy_path):
720             return FileResponse(proxy_path, media_type="video/mp4")
721
722         # 2) Otherwise resolve + serve the source.
723         try:
724             input_ref = storage.resolve_input_ref(
725                 rec["filepath"], getattr(cfg, "storage", None)
726             )
727         except ValueError as e: # unsupported scheme etc. ? a clean client error, never a
500
728             raise HTTPException(status_code=400, detail=str(e)) from e
729         except Exception as e: # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced
(not swallowed)
730             raise HTTPException(status_code=502, detail=f"Could not resolve the source
video: {e}") from e
731
732         if input_ref.backend == "local":
733             local = input_ref.key
734         else:
735             try:
736                 local = _cached_remote_source(video_id, input_ref, cfg)
737             except HTTPException:
738                 raise
739             except Exception as e: # noqa: BLE001 - a fetch miss is surfaced (not
swallowed)
740                 raise HTTPException(
741                     status_code=502, detail=f"Could not resolve the source video: {e}"
742                 ) from e
743         if not os.path.isfile(local):
744             raise HTTPException(status_code=404, detail="Source video file not found.")
745
746         # 3) Warm a proxy in the background for next time.
747         threading.Thread(
748             target=_warm_annotation_proxy, args=(video_id, local, cfg), daemon=True
749         ).start()

```

10. assistant (2026-07-02T08:16:46.535Z)

Master already implements the full intent of PR #418. Let me verify the supporting helpers (`\_remote\_input\_size\_bytes`, `require\_free\_bytes`, `resolve\_under\_root`) exist in master and confirm this came from #425.

11. user (2026-07-02T08:16:49.709Z)

6c767b5 cache remote source material
--- files touched by 6c767b5 (#425) ---

```
6c767b5 cache remote source material
backend/main.py | 76 ++++++-----
docs/KHELSUTRA_GURU_HEALTH_REMEDIATION_PLAN.md | 2 +-
tests/test_api_annotations.py | 44 ++++++
3 files changed, 114 insertions(+), 8 deletions(-)
```

12. user (2026-07-02T08:16:50.110Z)

```
backend\utils\freespace.py:49:def require_free_bytes(
backend\main.py:1479:def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
backend\utils\paths.py:45:def resolve_under_root(path: str, root: str) -> str:
```

13. assistant (2026-07-02T08:16:54.496Z)

Confirmed: #425 (commit `6c767b5 "cache remote source material"`) touched exactly the same two files (`backend/main.py`, `tests/test\_api\_annotations.py`) plus a docs line, and landed the same source_cache design. Let me verify the master tests to see whether #425's tests cover the same scenarios as #418's, and check the `\_remote\_input\_size\_bytes` helper for the free-space guard.

14. user (2026-07-02T08:16:55.442Z)

```
1479     def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
1480         """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
1481
1482         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
1483         that needs a
1484         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
1485         guard degrades
1486         to a no-op rather than blocking a job it cannot measure."""
1487         try:
1488             sc = (
1489                 storage_config.model_dump()
1490                 if hasattr(storage_config, "model_dump")
1491                 else (storage_config or {})
1492             )
1493             backend = storage.get_storage(input_ref.backend, config=sc)
1494             return int(backend.stat(input_ref).size)
1495         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
1496             not block
1497             logger.debug(
1498                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
1499             )
1500             return None
1501
1502     @app.post("/api/process", status_code=202)
```

15. user (2026-07-02T08:16:56.994Z)

```
29:def test_source_streams_the_recorded_video(tmp_path, monkeypatch):
41:def test_source_404_for_unknown_video(tmp_path, monkeypatch):
46:def test_source_404_when_the_file_is_gone(tmp_path, monkeypatch):
52:def test_source_serves_the_proxy_when_one_is_ready(tmp_path, monkeypatch):
67:def test_source_caches_remote_fetch_and_warms_proxy_from_cache(tmp_path, monkeypatch):
106:     cache_path = tmp_path / "source_cache" / "vidremote.mp4"
```

16. assistant (2026-07-02T08:17:00.717Z)

Let me read the master test that #425 added to compare coverage against #418's six tests.

17. user (2026-07-02T08:17:01.355Z)

```
67     def test_source_caches_remote_fetch_and_warms_proxy_from_cache(tmp_path, monkeypatch):
68         """P3: repeated source/Range requests for a remote video reuse one local source
```

```

cache."""
69         client, db = _client(tmp_path, monkeypatch)
70         db.add_video("vidremote", "gs://bucket/videos/match.mp4", "processed", [])
71         payload = b"REMOTE-video-bytes"
72         fetches = []
73         warmed = []
74
75         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
76             fetches.append((uri, dst))
77             with open(dst, "wb") as f:
78                 f.write(payload)
79             return dst
80
81         class _InlineThread:
82             def __init__(self, target, args=(), kwargs=None, daemon=None):
83                 self.target = target
84                 self.args = args
85                 self.kwargs = kwargs or {}
86
87             def start(self):
88                 self.target(*self.args, **self.kwargs)
89
90         monkeypatch.setattr(m.storage, "fetch", fake_fetch)
91         monkeypatch.setattr(m, "_remote_input_size_bytes", lambda *a, **k: len(payload))
92         monkeypatch.setattr(m.threading, "Thread", _InlineThread)
93         monkeypatch.setattr(
94             m,
95             "_warm_annotation_proxy",
96             lambda video_id, src_local, cfg: warmed.append((video_id, src_local)),
97         )
98
99         first = client.get("/api/source/vidremote", headers={"Range": "bytes=0-5"})
100        second = client.get("/api/source/vidremote", headers={"Range": "bytes=6-11"})
101
102        assert first.status_code in (200, 206)
103        assert second.status_code in (200, 206)
104        assert len(fetches) == 1
105        assert fetches[0][0] == "gs://bucket/videos/match.mp4"
106        cache_path = tmp_path / "source_cache" / "vidremote.mp4"
107        assert cache_path.read_bytes() == payload
108        assert warmed == [("vidremote", str(cache_path)), ("vidremote", str(cache_path))]
109
110
111        # ---- POST /api/annotations ----

```

18. assistant (2026-07-02T08:17:14.551Z)

I now have a complete picture. Let me do a final comparison of the two implementations' key design points to identify any material differences, and check the one meaningful behavioral difference: the cache filename scheme.

****Key comparison:****

Aspect	PR #418	Master (#425)
Fetch remote once into <code>`<output>/source_cache/`</code>	Yes	Yes (<code>`main.py:652-677`</code>)
Cache filename <code>`<video_id>/<sanitized-remote-name>`</code> (subdir per video) <code>`<video_id><ext>`</code> (flat, video_id-keyed) ?	<code>`main.py:643-649`</code>	
Atomic publish via temp + <code>`os.replace`</code>	Yes (<code>`partial`</code>)	Yes (<code>`tmp`</code>) ? <code>`main.py:665-670`</code>
Per-video lock + double-check	Yes	Yes (<code>`main.py:638-660`</code>)
Free-space guard (507 before fetch)	Yes	Yes (<code>`require_free_bytes`</code> , <code>`main.py:663-664`</code>)
Failed fetch cleans partial + surfaces 502	Yes	Yes (<code>`finally`</code> cleanup + 502 at <code>`main.py:738-741`</code>)
Local path identity passthrough (no fetch)	Yes	Yes (<code>`main.py:731-732`</code>)
Proxy warming for remote sources	Yes	Yes ? unconditional warm at <code>`main.py:746-748`</code>

```
| Path containment | `safe_output_filename` | `safe_output_filename` + `resolve_under_root`  
(stronger) ? `main.py:648-649` |
```

Master's #425 implements every element of #418's intent, and in a couple of respects is stronger (extra `resolve\_under\_root` containment, no unsafe remote-name-derived subdir). The only design difference is cosmetic: #418 stores under `source\_cache/<video\_id>/<remote-name>` while master stores `source\_cache/<video\_id><ext>` ? both achieve fetch-once-and-reuse keyed by video_id. The conflict is a direct same-region overlap in `backend/main.py` (the `\_source\_cache\_dir`/lock helpers, the `\_acquire\_source\_for\_serving` vs `\_cached\_remote\_source` block, and the rewritten `get\_source` body) plus `tests/test\_api\_annotations.py` (same test region). This is a full supersession, not adjacent edits.

19. user (2026-07-02T08:17:34.815Z)

Output does not match required schema: root: must have required property 'conflict_nature', root: must have required property 'correctness_notes', root: must have required property 'recommended_fate'

20. assistant (2026-07-02T08:17:38.215Z)

I had a malformed parameter block. Let me resubmit cleanly.

21. user (2026-07-02T08:17:51.340Z)

Structured output provided successfully